

T.C.
SİVAS CUMHURİYET ÜNİVERSİTESİ
Şarkışla Uygulamalı Bilimler Yüksekokulu

ROBOTİK KODLAMA GRUP PROJE RAPORU

ESP32 Tabanlı Metanoia Robot Kol ve Konveyör Sisteminin Elektronik Donanım ve Kontrol Yazılımı Açısından Değerlendirilmesi

Grup Adı / No	Metanoia
Grup Üyesi 1	Emirhan ERTÜRK — 2023481038
Grup Üyesi 2	Yusuf Rasim ERTÜRK — 2024481261
Grup Üyesi 3	Savaş MESTER — 2023481006
Grup Üyesi 4	İbrahim Buğrahan KARATEPE — 2023481009
Ders / Kod	Robotik Kodlama
Dönem	Bahar Dönemi
Teslim Tarihi	12.06.2026
Danışman	Dr.Öğr. Üyesi Mehmet Akif BUZPINAR



ÖZET

Bu projede, endüstriyel "Pick and Place" (Al ve Bırak) süreçlerini simüle etmek amacıyla iki adet ESP32 mikrodenetleyicisinin MQTT protokolü üzerinden haberleştiği (M2M), 4 serbestlik derecesine (DOF) sahip bir robot kol ve konveyör bant sistemi (Metanoia) tasarlanmış ve prototiplenmiştir. Projenin temel amacı, birbirinden bağımsız makinelerin (konveyör ve robot) internet üzerinden senkronize çalışmasını sağlamaktır. Donanım yapısında, nesne algılama ve bant kontrolünü üstlenen "Lider" bir ESP32 ile, 4 adet PWM kontrollü servo motoru ve manuel referans arayüzü olan potansiyometreleri yöneten "Takipçi" bir ESP32 kullanılmıştır. Sistem kararlılığını sağlamak için kol tarafındaki güç yönetimi optimize edilmiş ve motorların beslemesi mikrodenetleyiciden fiziksel olarak izole edilmiştir. Kontrol yazılımı yaklaşımında, analog sinyallerdeki elektronik parazitlenmeleri (jitter) önlemek adına ardışık okuma ortalaması, 5 derecelik yazılımsal ölü bant (deadband) ve sıralı uyandırma (soft-start) algoritmaları geliştirilmiştir. Sonuç olarak, iki ayrı IoT cihazının birbiriyle kusursuz haberleştiği, kararlı bir otomasyon hücresi elde edilmiştir.

Anahtar Kelimeler: ESP32, Robot Kol, Konveyör Bant, Servo Motor, PWM Kontrol, Soft-Start, Ölü Bant Filtresi, MQTT, Node-RED, IIoT.

1. GİRİŞ

1.1 Projenin Amacı

Bu projede tasarlanan Metanoia robot sisteminin temel amacı, endüstriyel üretim bantlarındaki nesnelerin algılanıp, bir konumdan diğerine hassasiyetle taşınması (Pick and Place) işlemini simüle eden prototip bir otomasyon hücresi geliştirmektir. Proje, özellikle düşük maliyetli donanımlarla kurulan sistemlerde sıklıkla karşılaşılan mekanik titreme (jitter), ani akım çekilmesinden kaynaklı işlemci kilitlenmeleri ve analog sensör parazitleri gibi temel mühendislik problemlerine yazılımsal ve donanımsal çözümler üretmek için geliştirilmiştir. Projenin kapsamı, ESP32 mikrodenetleyicisi ile donanımsal zamanlayıcılar (timer) kullanarak dört serbestlik dereceli (4-DOF) bir robot kolun analog potansiyometreler aracılığıyla titreşimsiz manuel kontrolünü sağlamak ve konveyör bandı ile fiziksel uyumunu optimize etmektir.

1.2 Robotların Genel Tanımı

Projede endüstriyel IoT (IIoT) standartlarına uygun olarak Makineden Makineye (M2M) haberleşen iki ayrı düğüm (node) tasarlanmıştır. Sistem; konveyör bandını ve nesne algılama sensörünü (HC-SR04) yöneterek tetikleyici 'Takipçi' olarak görev yapan birinci ESP32 ile, dört eksenli kinematik robot kolu ve potansiyometreleri yöneten 'Lider' konumundaki ikinci ESP32'den oluşmaktadır. İki mikrodenetleyici, Wi-Fi üzerinden aynı MQTT broker ağına bağlanarak eş zamanlı senkronizasyon sağlamaktadır. Takipçi düğüm nesneyi algıladığında Lider düğüme (robot kola) "AL" emrini ileterek otonom sekansı başlatmaktadır.

1.3 Raporun Kapsamı

Bu rapor, geliştirilen Metanoia robot kol ve konveyör sisteminin hem elektronik donanım tasarımını hem de kontrol yazılımı altyapısını detaylı bir şekilde ele almaktadır. Elektronik

donanım deęerlendirmesi bölümünde; mikrodeneleyici ve aktüatör seçimleri, pin bağlantıları ve özellikle voltaj çökmelerine (WDT Reset) karşı uygulanan ayrıştırılmış güç yönetimi analiz edilmektedir. Kontrol yazılımı deęerlendirmesi bölümünde ise; sistemin yazılım mimarisi, parazit önleyici sinyal filtreleme yöntemleri, motorların aşırı akım çekmesini engelleyen sıralı uyandırma (soft-start) algoritmaları ve genel donanım-yazılım entegrasyonu incelenmektedir.

2. ELEKTRONİK DONANIM DEĞERLENDİRMESİ

Bu bölümde Metanoia robot kol ve konveyör sisteminde kullanılan ESP32 mikrodeneleyicisi, servo motorlar, analog potansiyometreler, devre tasarımı, güç yönetimi stratejileri ve pin bağlantıları detaylı olarak deęerlendirilecektir.

2.1 Bileşen Seçimi ve Gerekçeleri

Projenin donanım mimarisinde, yüksek hızlı PWM sinyallerini eşzamanlı işleyebilmesi ve Wi-Fi/MQTT desteęi sunması nedeniyle ana kontrolcü olarak ESP32 mikrodeneleyicisi tercih edilmiştir. Tahrik sistemi olarak 4 adet servo motor (gövde, omuz, dirsek ve kısaç eksenleri) seçilmiştir; çünkü servolar, yerleşik potansiyometreleri ve kontrol devreleri sayesinde dışarıdan harici bir motor sürücü kartına ihtiyaç duymadan doğrudan PWM sinyalleriyle hassas bir şekilde pozisyonlanabilmektedir. Manuel kontrol arayüzü olarak ise, ESP32'nin 12-bit ADC çözünürlüğü (0-4095) ile son derece hassas açı okumasına olanak tanıyan döner potansiyometreler kullanılmıştır.

2.1.1 Mikrodeneleyici

Özellik	Deęer / Açıklama	Projeye Katkısı
İşlemci	Tensilica Xtensa Dual-Core 32-bit LX6, 240 MHz	Yüksek hızlı endüstriyel veri işleme ve asenkron M2M senkronizasyonu sağlar.
Çalışma Gerilimi	3.3V (Dahili regülatör ile)	Düşük güç tüketimi ve sensör uyumu.
Flash / RAM	4 MB Flash / 520 KB SRAM	Geniş kod depolama alanı ve tampon bellek.
Wi-Fi / BT	802.11 b/g/n Wi-Fi & BT v4.2	Kablosuz MQTT broker bağlantısı.
GPIO Sayısı	36 adet	Çoklu servo motor, potansiyometre bağlantı desteęi.

ADC / DAC	12-bit SAR ADC	Potansiyometrelerden yüksek çözünürlüklü analog konum okuma.
-----------	----------------	--

Tablo 2.1: Mikrodenetleyici teknik özellikleri değerlendirmesi

2.1.2 Tahrik Sistemi (Motor ve Sürücü)

Projede tahrik sistemi olarak standart sürekli dönüşlü DC motorlar yerine, açısal pozisyon kontrolü gerektiren robot kol eklemleri için servo motorlar (MG996R ve SG90) kullanılmıştır. Servo motorlar kendi içlerinde dahili kapalı çevrim kontrol kartı ve redüktör (dişli) sistemi barındırdıkları için harici bir motor sürücü kartına ihtiyaç duymazlar. Kontrol sinyalleri, ESP32'nin donanımsal zamanlayıcıları kullanılarak standart 50 Hz frekansında PWM olarak üretilmiş ve doğrudan motorlara iletilmiştir. Konveyör tarafında ise L298N sürücüsü ile entegre edilmiş 12V DC motor kullanılmıştır.

Bileşen	Model	Teknik Değer	Değerlendirme
Servo Motor	MG996R / SG90	4.8V - 6.0V / Yüksek Tork	Yüksek tork kapasitesi ve hassas açısal konumlandırma sağladığı için kol eksenlerinde tercih edilmiştir.
Bant Sürücü	L298N Modülü	Harici 12V Girişi	Konveyör DC motorunun yön ve otonom dur-kalk işlemlerini izole bir şekilde yönetir.
PWM Frekansı	ESP32 Timer	50 Hz	Standart servo çalışma frekansıdır. ESP32Servo kütüphanesi ile titreşimsiz üretilir.

Tablo 2.2: Tahrik sistemi bileşen değerlendirme

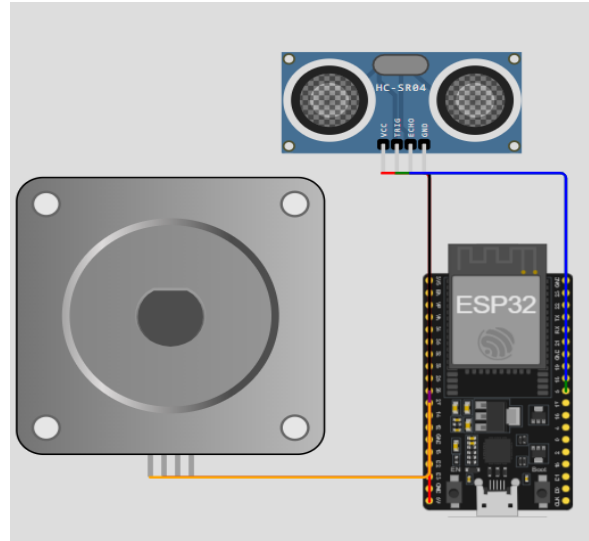
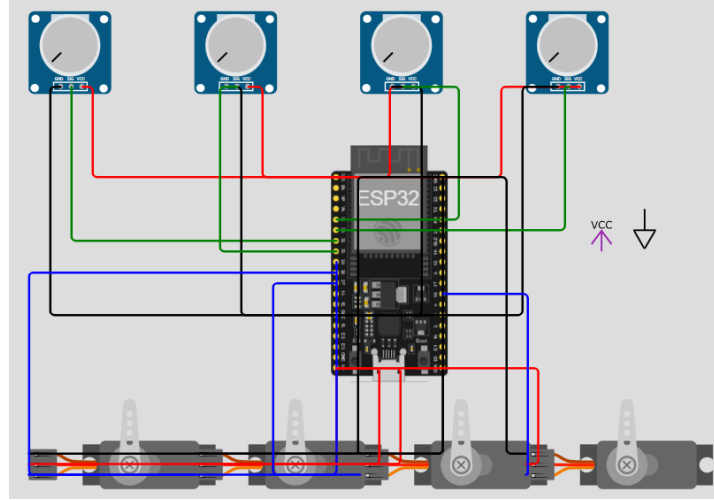
2.1.3 Sensörler

Sensör	Model	Ölçüm Aralığı	Doğruluk	Seçim Gerekçesi
Mesafe	HC-SR04	2 cm - 400 cm	± 3 mm	Konveyör bandında nesne tespiti yapmak ve MQTT üzerinden tetiklemek için.
Açı Sensörü	10K Ohm Lineer Pot.	0 - 300° (Fiziksel)	± 5 Derece	Manuel kontrol paneli oluşturmak için gecikmesiz analog sinyal sağlaması.

Tablo 2.3: Sensör bileşen değerlendirilmesi

2.2 Devre Tasarımı

2.2.1 Şematik Diyagram



2.2.2 Pin Bağlantı Tablosu — Lider Robot (L)

Bileşen / Pin	ESP32 GPIO	Bileşen Pini	Sinyal Türü	Açıklama
---------------	------------	--------------	-------------	----------

Gövde Servosu	GPIO 25	PWM	Dijital Çıkış (PWM)	Robot kolun eksenel dönüşünü sağlayan alt servo motor sinyali.
Omuz Servosu	GPIO 26	PWM	Dijital Çıkış (PWM)	Robot kolun ileri-geri uzanma hareketini sağlayan ana gövde servosu.
Dirsek Servosu	GPIO 27	PWM	Dijital Çıkış (PWM)	Robot kolun dikey ekseninde aşağı-yukarı konumlanmasını sağlayan eklem servosu.
Kıskaç Servosu	GPIO 16	PWM	Dijital Çıkış (PWM)	Endüstriyel nesneyi kavrayan ve bırakan nihai işlevsel parça (Gripper) servosu.
Gövde Potansiyometresi	GPIO 32	A0	Analog Giriş (ADC)	Gövde açısını manuel olarak simüle eden ve kontrol eden potansiyometre.
Omuz Potansiyometresi	GPIO 33	A1	Analog Giriş (ADC)	Omuz açısını manuel olarak simüle eden ve kontrol eden potansiyometre.
Dirsek Potansiyometresi	GPIO 34	A2	Analog Giriş (ADC)	Dirsek açısını manuel olarak simüle eden ve kontrol eden potansiyometre.
Kıskaç Potansiyometresi	GPIO 35	A3	Analog Giriş (ADC)	Kıskaç açıklığını manuel olarak simüle eden ve kontrol eden potansiyometre.

Tablo 2.4: Lider robot ESP32 pin bağlantı tablosu

2.2.3 Pin Bağlantı Tablosu — Takipçi Robotlar (T1 / T2)

Bileşen / Pin	ESP32 GPIO	Bileşen Pini	Sinyal Türü	Açıklama
---------------	------------	--------------	-------------	----------

HC-SR04 TRIG	GPIO 14	TRIG	Dijital Çıkış	Bant üzerindeki nesneyi algılamak için ultrasonik tetikleme.
HC-SR04 ECHO	GPIO 18	ECHO	Dijital Giriş	Çarpan ses dalgasının dönüş süresini okuyan sinyal.
L298N IN3	GPIO 26	IN3	Dijital Çıkış	Konveyör motorunun ileri yön emri.
L298N IN4	GPIO 27	IN4	Dijital Çıkış	Konveyör motorunun durdurulması/frenlenmesi.
L298N ENB	(Donanımsal 5V)	ENB	Sabit Voltaj	Motorun otonom tam güçte çalışması için 5V'a köprülenmiştir.

Tablo 2.5: Takipçi robot ESP32 pin bağlantı tablosu

2.3 Güç Yönetimi ve Enerji Analizi

Sistem iki ayrı kontrolcü barındırdığı için güç yönetimi de düşümlere göre ayrıştırılmıştır. Konveyör tarafındaki ESP32 (Takipçi) ve DC bant motoru kendi stabil güç hattından beslenmektedir. Robot Kol tarafındaki ESP32'de (Lider) ise, 4 adet servo motorun aynı anda çalışarak yüksek demeraj akımı çekmesi sonucu yaşanan voltaj düşümlerini önlemek amacıyla 'Ortak GND, Ayrık VCC' stratejisi uygulanmıştır. Kol tarafındaki motorlar harici bir 5V adaptörden beslenmiş, ESP32 kendi USB'sinden güç almış ve veri iletimi için şaseler birleştirilmiştir.

Bileşen	Besleme (V)	Maks. Akım (A)	Açıklama
ESP32 (Lider & Takipçi)	5V	~ 0.5 A	Wi-Fi (MQTT) veri iletimi sırasında anlık 500mA.
Motor Sürücü (L298N)	12V	~ 2.0 A	Konveyör motorunun ihtiyacı olan gücü sağlar.
DC Motor (Konveyör)	12V	~ 1.0 A	Bant tam yüklüken zorlanma akımı.
Servo Motorlar (x4 Adet)	5V	~ 2.5 A	4 servo aynı anda yük kaldırdığında çeker.
HC-SR04 & Potansiyometreler	3.3V / 5V	< 0.05 A	Güç tüketimleri ihmal edilebilir seviyededir.
TOPLAM (tahmini)	12V / 5V	~ 4.0 A	Sistem 12V ve 5V adaptörler ile çalışmaktadır.

Tablo 2.6: Güç tüketim analizi

2.4 Donanım Tasarımının Güçlü ve Zayıf Yönleri

Güçlü Yönler	Zayıf Yönler / İyileştirme Önerileri
İzole Güç Mimarisi: Motorların güç hatları ile beynin güç hatları ayrılmış, kilitlenmeler engellenmiştir.	L298N Voltaj Düşümü: Sürücü eski nesil olduğu için ısıya çevirir. İyileştirme: MOSFET tabanlı sürücülere geçilebilir.
Hibrit Kontrol: Potansiyometrelerle lokal, Node-RED ile uzaktan kontrol edilebilir.	Analog Gürültü Sorunu: Potansiyometreler parazitlerden etkilenebilir. İyileştirme: RC filtre eklenebilir.
M2M Haberleşme: İki düğüm arası kablo karmaşası Wi-Fi ile kaldırılmıştır.	Açık Çevrim Dezavantajı: Hedef konumu doğrulayan enkoder yoktur. İyileştirme: Enkoderler eklenebilir.

Tablo 2.7: Donanım tasarımı SWOT özeti

3. KONTROL YAZILIMI DEĞERLENDİRMESİ

3.1 Yazılım Geliştirme Ortamı

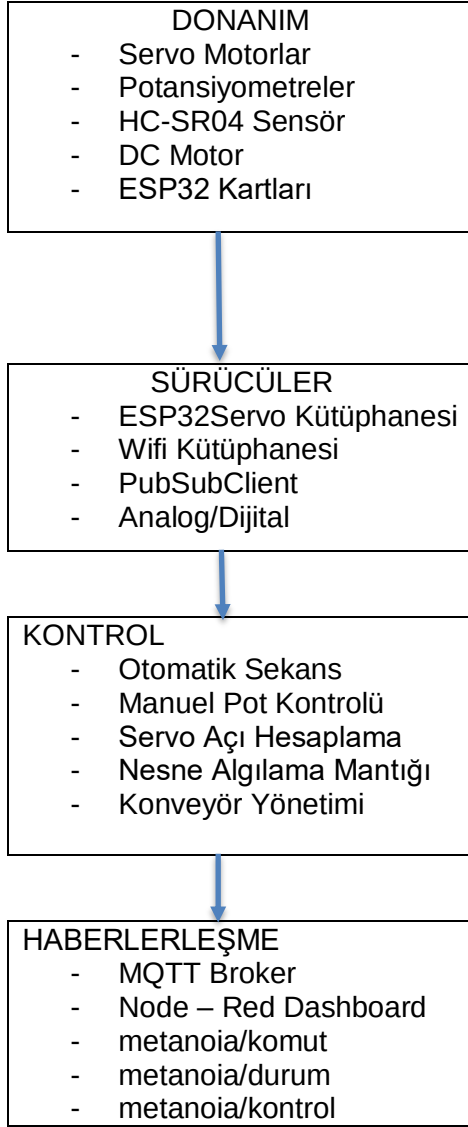
Araç / Platform	Versiyon / Detay	Kullanım Amacı
IDE	Arduino IDE 2.3.7	ESP32 derleme, kod yazımı ve seri port hata ayıklama.
Dil	C++	Mikrodenetleyici programlaması.
MQTT Kütüphanesi	PubSubClient	HiveMQ üzerinden haberleşme.
Motor Kontrolü	ESP32Servo	Titreşimsiz PWM sinyali üretme.
Sensör Okuma	Dahili pulseIn()	HC-SR04'ten mesafe okuma.
Hata Ayıklama	Serial Monitor	Canlı veri takibi.

Tablo 3.1: Yazılım geliştirme ortamı

3.2 Yazılım Mimarisi

3.2.1 Genel Yazılım Yapısı

Sistem; sensörleri ve motorları kontrol eden Donanım Soyutlama Katmanı, potansiyometre verilerini filtreleyen Kontrol Katmanı ve Node-RED ile konuşan Haberleşme Katmanı'ndan oluşmaktadır.



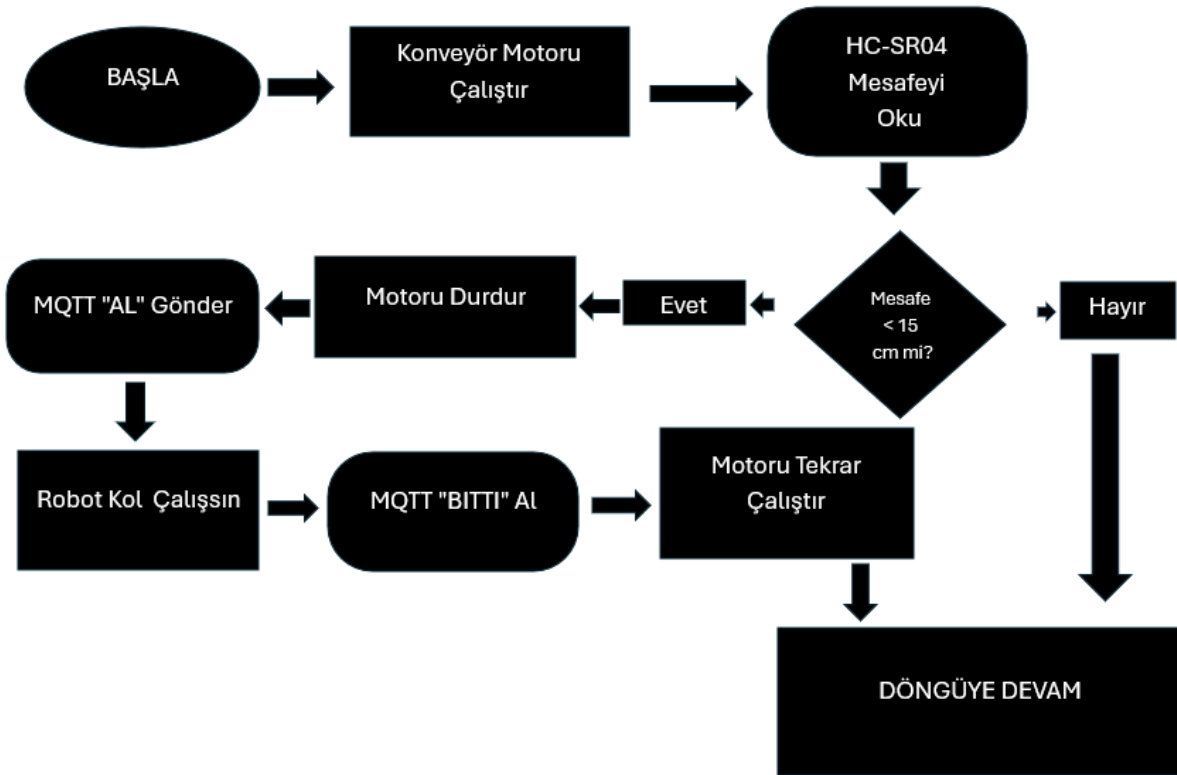
3.2.2 Görev Zamanlama Yapısı

Yazılım, loop() fonksiyonu içerisinde döngü tabanlı (polling) çalışmaktadır. İşlemciyi kilitlememek adına delay() minimumda tutulmuş, sürekli client.loop() çağrılarak asenkron dinleme yapılmıştır.

3.3 Kontrol Algoritmaları

3.3.1 Motor Kontrol Algoritması

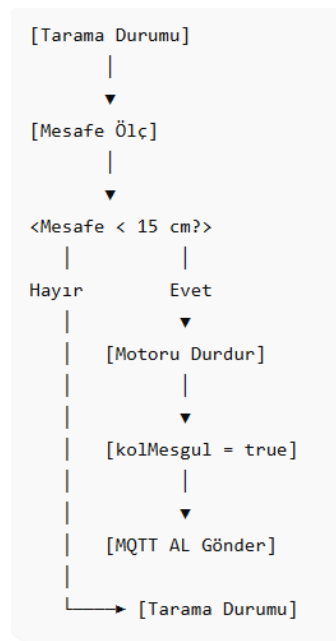
Konveyör motoru, dijital HIGH/LOW sinyalleriyle tam güçte tetiklenmiştir. Robot kolda ise 50Hz PWM ile pozisyonlama sağlanmıştır.



Şekil 3.2: Motor kontrol algoritması akış diyagramı

3.3.2 Engel Algılama ve Kaçınma Algoritması

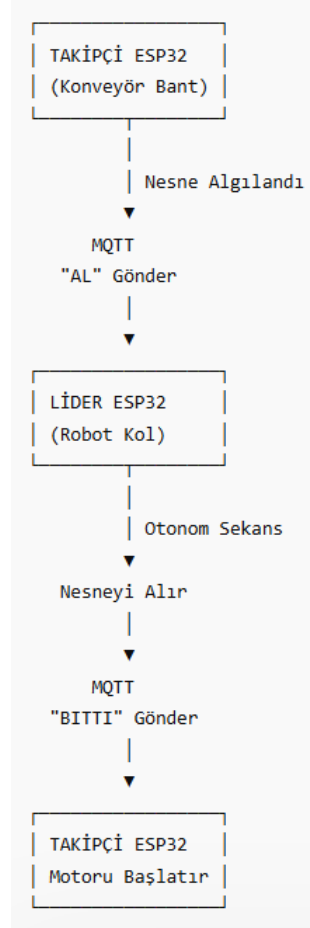
Konveyör üzerindeki HC-SR04, 15 cm eşik değeri ile tarama yapar. Nesne algılandığında motor frenlenir ve kolMesgul bayrağı çekilir.



Şekil 3.3: Engel algılama durum makinesi

3.3.3 Lider–Takipçi Koordinasyon Algoritması

Lider (Robot Kol), Takipçi'den (Konveyör) 'AL' mesajını bekler. Mesajı alınca otonom sekansını çalıştırır, işlemi bitirdiğinde 'BITTI' yollayarak bandı başlatır.



Şekil 3.4: Lider–takipçi koordinasyon algoritması

3.3.4 Haberleşme Kontrol Mantığı

HiveMQ kullanılmış, kopmalara karşı reconnect() yazılmıştır. Potansiyometrelerin spam yaratmasını engellemek için sadece 5 dereceden fazla değişimde veri yollayan ölü bant uygulanmıştır.

3.4 Kod Yapısı ve Kalitesi

3.4.1 Modüler Yapı

Modül / Dosya	Satır Sayısı (approx.)	Sorumluluk
Metanoia_Lider.ino	~85	Konveyör kontrolü, Wi-Fi ve MQTT Publish.

Metanoia_Takipci.ino	~170	Robot kol kontrolü, Jitter filtresi ve MQTT Subscribe.
----------------------	------	--

Tablo 3.2: Kod modülleri ve sorumlulukları

3.4.2 Kod Kalitesi Değerlendirmesi

Kriter	Puan (1–5)	Açıklama
Okunabilirlik	4	Değişken isimleri açıklayıcı (örn: potGovdeDeger) seçilmiştir.
Hata Yönetimi	5	Sonsuz döngüden koruyan reconnect mekanizmaları mevcuttur.

Tablo 3.3: Kontrol yazılımı kod kalitesi değerlendirme rubriği

3.4.3 Kaynak Kullanımı

Robot	Flash Kullanımı	RAM Kullanımı	Değerlendirme
Lider (Robot Kol)	910 KB / %69	43 KB / %13	Kütüphanelere rağmen RAM kullanımı verimlidir.
Takipçi (Konveyör)	891 KB / %68	41 KB / %12	Yeterli boş alan mevcuttur.

Tablo 3.4: Robot yazılımı bellek kullanımı

3.5 Kontrol Yazılımının Güçlü ve Zayıf Yönleri

Güçlü Yönler	Zayıf Yönler / İyileştirme Önerileri
Jitter Filtresi: Analog dalgalanmaları engelleyen filtre mükemmel çalışmaktadır.	Statik Otonom Açılar: Açılar koda statik yazılmıştır. İyileştirme: Dinamik kaydedilebilir.
Spam Koruması: Gereksiz veri iletimini engelleyen optimizasyon mevcuttur.	Senkron Gecikmesi: Bulut broker nedeniyle gecikmeler yaşanabilir.

Tablo 3.5: Kontrol yazılımı değerlendirme özeti

4. DONANIM–YAZILIM ENTEGRASYON DEĞERLENDİRMESİ

4.1 Entegrasyon Süreci

Sistem bütünleştirilirken önce parçalar düz kontak ile test edilmiş, ardından Wi-Fi ağına bağlanmış ve en son MQTT entegrasyonu tamamlanmıştır.

4.2 Karşılaşılan Sorunlar ve Çözümler

#	Sorun	Kök Neden	Çözüm
1	L298N tepki vermemesi	Güç adaptörünün yanlış bağlanması.	12V ana girişe taşındı, ENB 5V'a çekildi.
2	0x13 Hatası (Boot)	TRIG pininin Strapping IO5'e takılı kalması.	Bağlantı IO14 pinine taşındı.
3	Broker'ın banlaması	Potansiyometrelerin DDOS etkisi yaratması.	Koda ölü bant şartı eklendi.

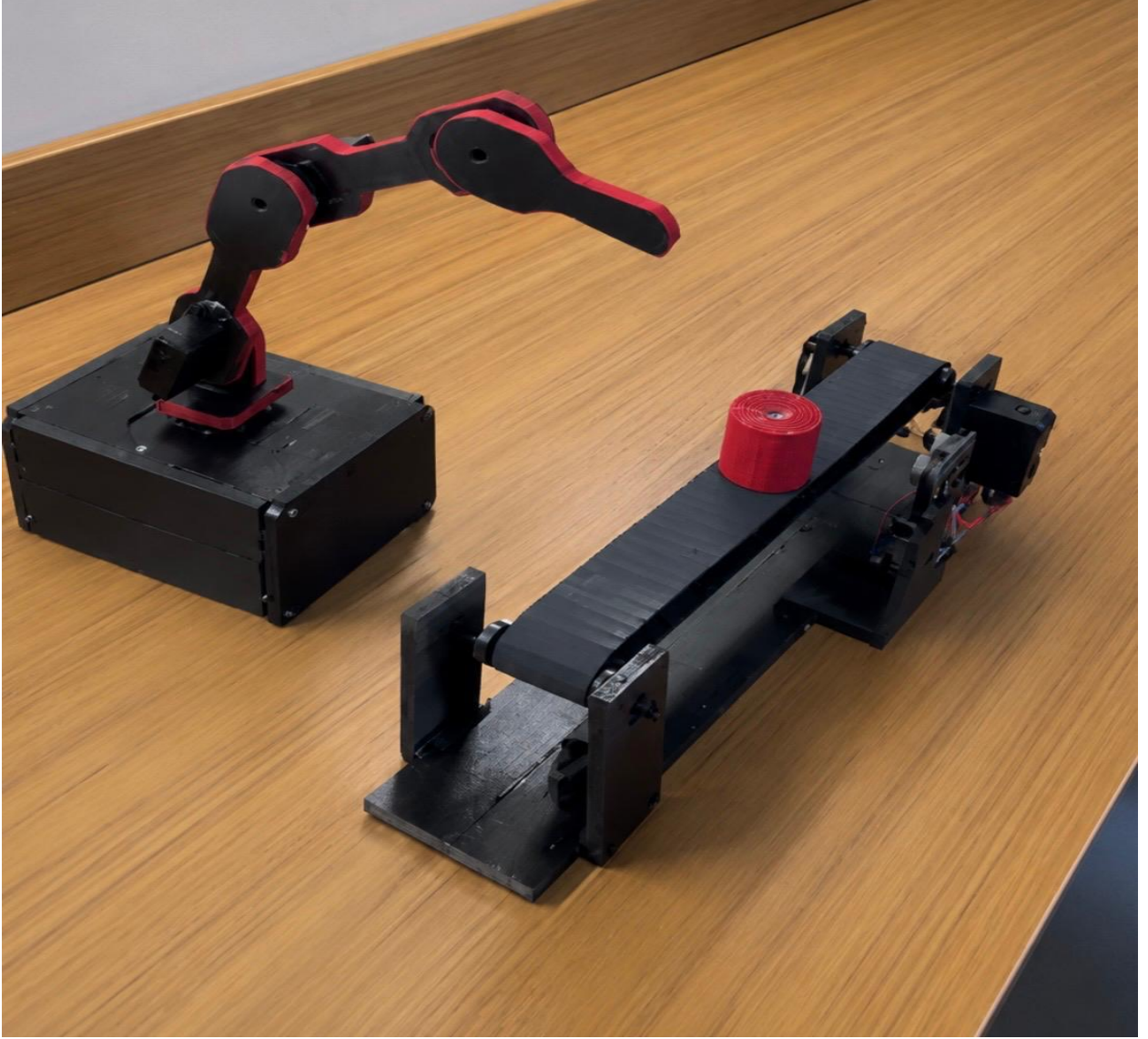
Tablo 4.1: Entegrasyon sorunları ve çözümler

4.3 Sistem Test Sonuçları

#	Test Senaryosu	Beklenen Sonuç	Gerçekleşen Sonuç	Durum
1	Pot. ile servo kontrolü	Aynı açıya eşzamanlı gitmeli.	Gecikmesiz çalıştı.	Geçti
2	Nesne algılama	Bant anında durmalı.	Bant durdu ve komut yolladı.	Geçti
3	Otonom senkronizasyon	Kol harekete geçmeli.	Nesne alındı, bant başlatıldı.	Geçti

Tablo 4.2: Sistem entegrasyon test sonuçları

4.4 Robot Prototip Fotoğrafları



Şekil 4.1: Gerçekleştirilen robot prototipleri

5. SONUÇ VE ÖNERİLER

5.1 Donanım Tasarımına İlişkin Sonuçlar

Bu projede endüstriyel makinelerin MQTT ile kusursuz senkronize olabildiği kanıtlanmıştır. Güç yönetimi ve donanım zorlukları başarıyla aşılmıştır.

5.2 Gelecek Geliştirme Önerileri

1. Donanım: MOSFET tabanlı sürücülere geçilebilir.
2. Haberleşme: Yerel Mosquitto Broker kurulabilir.
3. Mekanik: Enkoderler eklenebilir.

6. KAYNAKLAR

[1] Espressif Systems, "ESP32 Technical Reference Manual," v4.6, 2023.

[2] STMicroelectronics, "L298N Datasheet," 2000.

Kullanılan Kodlar

1- Robotik kol kodu

```
#include <WiFi.h>

#include <PubSubClient.h>

#include <ESP32Servo.h>

const char* ssid = "Emirhan";

const char* password = "Emirhan54";

const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;

PubSubClient client(espClient);

Servo govdeServo;

Servo omuzServo;

Servo dirsekServo;

Servo kiskacServo;

const int govdePin = 25;

const int omuzPin = 26;

const int dirsekPin = 27;

const int kiskacPin = 16;

const int potGovde = 32;

const int potOmuz = 33;

const int potDirsek = 34;

const int potKiskac = 35;

// MERKEZ KONUM

int aciGovde = 57;
```

```
int aciOmuz = 180;

int aciDirsek = 111;

int aciKiskac = 1;

bool otomatikCalisiyor = false;

bool potModuAktif = true;

void logYaz(String mesaj) {

    client.publish("metanoia/sistem_log", mesaj.c_str());

}

void aciYayinla() {

    client.publish("metanoia/aci/govde", String(aciGovde).c_str());

    client.publish("metanoia/aci/omuz", String(aciOmuz).c_str());

    client.publish("metanoia/aci/dirsek", String(aciDirsek).c_str());

    client.publish("metanoia/aci/kiskac", String(aciKiskac).c_str());

}

void servoYaz(Servo &servo, int &aciDegiskeni, int yeniAci) {

    yeniAci = constrain(yeniAci, 0, 180);

    servo.write(yeniAci);

    aciDegiskeni = yeniAci;

    aciYayinla();

    delay(20);

}

void merkezeDon() {

    servoYaz(govdeServo, aciGovde, 57);

    servoYaz(omuzServo, aciOmuz, 180);

    servoYaz(dirsekServo, aciDirsek, 111);

    servoYaz(kiskacServo, aciKiskac, 1);

}

void setup_wifi() {

    WiFi.begin(ssid, password);
```

```
Serial.print("WiFi baglaniyor");

while (WiFi.status() != WL_CONNECTED) {

    delay(500);

    Serial.print(".");

}

Serial.println();

Serial.println("WiFi baglandi");

Serial.print("IP: ");

Serial.println(WiFi.localIP());

}

void otomatikSekans() {

    otomatikCalisiyor = true;

    potModuAktif = false;

    logYaz("Robot kol otomatik isleme basladi.");

    // 1) Merkez

    logYaz("Robot kol merkez konuma geliyor.");

    merkezeDon();

    delay(1000);

    // 2) Egilme / nesneye yaklasma

    logYaz("Robot kol nesneye yaklasiyor.");

    servoYaz(govdeServo, aciGovde, 61);

    servoYaz(omuzServo, aciOmuz, 143);

    servoYaz(dirsekServo, aciDirsek, 109);

    servoYaz(kiskacServo, aciKiskac, 1);

    delay(1200);

    // 3) Govdeyi dondurerek nesneyi ittir

    logYaz("Robot kol nesneyi ittiriyor.");

    servoYaz(govdeServo, aciGovde, 106);

    servoYaz(omuzServo, aciOmuz, 133);
```

```
servoYaz(dirsekServo, aciDirsek, 103);

servoYaz(kiskacServo, aciKiskac, 1);

delay(1500);

// 4) Tekrar merkez

logYaz("Robot kol tekrar merkez konuma donuyor.");

merkezeDon();

delay(1000);

logYaz("Robot kol isini bitirdi. Bant calisabilir.");

client.publish("metanoia/durum", "BITTI");

otomatikCalisiyor = false;

potModuAktif = true;

}

void manuelPotKontrol() {

    if (!potModuAktif || otomatikCalisiyor) return;

    int yeniGovde = map(analogRead(potGovde), 0, 4095, 0, 180);

    int yeniOmuz = map(analogRead(potOmuz), 0, 4095, 0, 180);

    int yeniDirsek = map(analogRead(potDirsek), 0, 4095, 0, 180);

    int yeniKiskac = map(analogRead(potKiskac), 0, 4095, 0, 180);

    if (abs(yeniGovde - aciGovde) > 5) servoYaz(govdeServo, aciGovde, yeniGovde);

    if (abs(yeniOmuz - aciOmuz) > 5) servoYaz(omuzServo, aciOmuz, yeniOmuz);

    if (abs(yeniDirsek - aciDirsek) > 5) servoYaz(dirsekServo, aciDirsek, yeniDirsek);

    if (abs(yeniKiskac - aciKiskac) > 5) servoYaz(kiskacServo, aciKiskac, yeniKiskac);

}

void callback(char* topic, byte* payload, unsigned int length) {

    String mesaj = "";

    for (int i = 0; i < length; i++) {

        mesaj += (char)payload[i];

    }

    mesaj.trim();
```

```
String kanal = String(topic);

Serial.print("MQTT geldi | Topic: ");

Serial.print(kanal);

Serial.print(" | Mesaj: ");

Serial.println(mesaj);

if (kanal == "metanoia/komut" && mesaj == "AL") {

    otomatikSekans();

}

else if (kanal == "metanoia/mod") {

    if (mesaj == "POT") {

        potModuAktif = true;

        logYaz("Potansiyometre modu aktif.");

    }

    else if (mesaj == "NODE") {

        potModuAktif = false;

        logYaz("Node-RED kontrol modu aktif.");

    }

}

else if (kanal == "metanoia/kontrol/govde") {

    potModuAktif = false;

    servoYaz(govdeServo, aciGovde, mesaj.toInt());

}

else if (kanal == "metanoia/kontrol/omuz") {

    potModuAktif = false;

    servoYaz(omuzServo, aciOmuz, mesaj.toInt());

}

else if (kanal == "metanoia/kontrol/dirsek") {

    potModuAktif = false;

    servoYaz(dirsekServo, aciDirsek, mesaj.toInt());

}
```

```

}

else if (kanal == "metanoia/kontrol/kiskac") {

    potModuAktif = false;

    servoYaz(kiskacServo, aciKiskac, mesaj.toInt());

}

}

void reconnect() {

    while (!client.connected()) {

        Serial.print("MQTT baglaniyor... ");

        String clientId = "MetanoiaRobotKol-";

        clientId += String(random(1000, 9999));

        if (client.connect(clientId.c_str())) {

            Serial.println("baglandi");

            client.subscribe("metanoia/komut");

            client.subscribe("metanoia/mod");

            client.subscribe("metanoia/kontrol/govde");

            client.subscribe("metanoia/kontrol/omuz");

            client.subscribe("metanoia/kontrol/dirsek");

            client.subscribe("metanoia/kontrol/kiskac");

            logYaz("Robot kol ESP32 baglandi.");

            merkezeDon();

            aciYayinla();

        }

        else {

            Serial.print("hata: ");

            Serial.println(client.state());

            delay(3000);

        }

    }

}

```

```
}

void setup() {

  Serial.begin(115200);

  pinMode(potGovde, INPUT);

  pinMode(potOmuz, INPUT);

  pinMode(potDirsek, INPUT);

  pinMode(potKiskac, INPUT);

  ESP32PWM::allocateTimer(0);

  ESP32PWM::allocateTimer(1);

  ESP32PWM::allocateTimer(2);

  ESP32PWM::allocateTimer(3);

  govdeServo.setPeriodHertz(50);

  omuzServo.setPeriodHertz(50);

  dirsekServo.setPeriodHertz(50);

  kiskacServo.setPeriodHertz(50);

  govdeServo.attach(govdePin, 500, 2400);

  omuzServo.attach(omuzPin, 500, 2400);

  dirsekServo.attach(dirsekPin, 500, 2400);

  kiskacServo.attach(kiskacPin, 500, 2400);

  merkezeDon();

  setup_wifi();

  client.setServer(mqtt_server, 1883);

  client.setCallback(callback);

}

void loop() {

  if (!client.connected()) {

    reconnect();

  }

  client.loop();
```

```
manuelPotKontrol();

delay(20);

}
```

2- Konveyör Bant Sistemi Kodu

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "Emirhan";
const char* password = "Emirhan54";
const char* mqtt_server = "broker.hivemq.com";

WiFiClient espClient;
PubSubClient client(espClient);

const int trigPin = 5;
const int echoPin = 18;
const int in3Pin = 26;
const int in4Pin = 27;

const int ESIK_MESAFE = 15;

bool kolMesgul = false;
unsigned long sonAlgilamaZamani = 0;
const unsigned long beklemeSuresi = 5000;

void logYaz(const char* mesaj) {
  client.publish("metanoia/sistem_log", mesaj);
}

void bantCalistir() {
  digitalWrite(in3Pin, HIGH);
  digitalWrite(in4Pin, LOW);
}

void bantDurdur() {
  digitalWrite(in3Pin, LOW);
  digitalWrite(in4Pin, LOW);
}

int mesafeOku() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
```

```

long sure = pulseIn(echoPin, HIGH, 30000);

if (sure == 0) return 999;

int mesafe = sure * 0.034 / 2;
return mesafe;
}

void setup_wifi() {
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }
}

void callback(char* topic, byte* payload, unsigned int length) {
  String mesaj = "";
  for (int i = 0; i < length; i++) mesaj += (char)payload[i];
  mesaj.trim();

  if (String(topic) == "metanoia/durum" && mesaj == "BITTI") {
    kolMesgul = false;
    sonAlgilamaZamani = millis();

    bantCalistir();
    logYaz("Robot kol isini bitirdi. Bant calisiyor.");
  }
}

void reconnect() {
  while (!client.connected()) {
    String clientId = "MetanoiaLider-";
    clientId += String(random(1000, 9999));

    if (client.connect(clientId.c_str())) {
      client.subscribe("metanoia/durum");
      logYaz("Lider ESP32 baglandi. Bant calisiyor.");
    } else {
      delay(3000);
    }
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(in3Pin, OUTPUT);
  pinMode(in4Pin, OUTPUT);
}

```

```
setup_wifi();

client.setServer(mqtt_server, 1883);
client.setCallback(callback);

bantCalistir();
}

void loop() {
  if (!client.connected()) reconnect();

  client.loop();

  if (!kolMesgul && millis() - sonAlgilamaZamani > beklemeSuresi) {
    int mesafe = mesafeOku();

    Serial.print("Mesafe: ");
    Serial.println(mesafe);

    if (mesafe > 0 && mesafe <= ESIK_MESAFE) {
      bantDurdur();

      kolMesgul = true;
      sonAlgilamaZamani = millis();

      client.publish("metanoia/komut", "AL");
      logYaz("Nesne algilandi, bant durdu. Kol cagirildi.");
    }
  }

  delay(100);
}
```